

Operit2 技术白皮书

个人设备空间中的持续 Agent 核心

第一版

作者 Operit 团队

2026 年 9 月



摘要

Operit2 面向一个长期存在的问题：人的设备越来越多，能力越来越分散，而 Agent 的上下文、行动和结果往往仍被锁在一台机器、一个窗口或一次短暂会话中。我们希望把手机、桌面、浏览器、云端设备以及未来可接入的硬件，组织成属于同一位用户的个人设备空间。每台设备保留自己的系统能力和权限边界，Agent 则在它们之间保持任务、上下文与使用者意图的连续性。

这份白皮书描述 Operit2 的工程判断和长期方向。它的核心贡献不是宣称替代所有 Agent，也不是把所有设备压缩成同一种运行环境。我们关心的是：当模型、提示词、工具规划和交互形态不断改变时，用户的对话、任务、数据、设备关系和授权决策能否持续存在。为此，Operit2 将可快速变化的智能层设计为可替换部分，同时把节点身份、Host 能力、持久化记录、同步语义和用户主权作为更稳定的基础设施。

本文以 Operit2 的本地源码、架构文档和公开仓库为工程基线，并借助少量 Agent、行动轨迹和外部记忆研究来说明设计位置。研究引用用于建立共同语言，不构成性能比较、行业排名或对他人的评价。

内容目录

- 一 项目起点与问题重构
- 二 面向个人的 Agent 设备空间
- 三 Operit2 的工程架构
- 四 对话主权 任务执行与上下文连续性
- 五 设备能力 健康与调度
- 六 权限 同步与用户主权
- 七 插件化与可演进的运行时
- 八 研究语境与技术立场
- 九 可靠性 迁移与预览期演进
- 十 实施重点与结语
- 附录 源码基线与参考资料

项目起点与问题重构

Operit 的第一代产品从 Android Agent 出发。团队没有把手机当成一个只负责显示对话的薄客户端，而是把本地 Linux 执行环境、终端会话、文件系统和移动端权限能力一起带进 Agent 的工作范围。历史代码中可以看到 PRoot 驱动的 Ubuntu 环境、可见 PTY 会话、后台复用 shell，以及通过 Android Storage



Access Framework 暴露 Ubuntu 文件系统的实现。这段经历让我们很早面对一个现实：设备的形态不应决定它在 Agent 世界里的价值。[2]

当移动端 Agent 的能力被推到较深的位置后，Operit2 最初试图解决的是跨平台扩展。然而，单纯让同一套对话和工具出现在更多终端，并不能回答用户真正会遇到的问题。手机始终在线，却不适合承载所有重任务；桌面适合长时间创作和审核，却不在用户身边；一台云端设备可能连续工作很久，却不应因此取得身份上的特权。我们因此重新定义了问题：Agent 不应只属于某个客户端，它应该能够在一个人的设备之间持续存在。

这种问题重构也决定了 Operit2 的产品范围。我们优先服务个人，而不是把设备协作改造成组织治理系统。一个人可以拥有多台设备，也可以把云服务器、浏览器或新购买的电脑加入自己的空间；但设备协作不等于多人审批链、企业组织结构或管理层级。Operit2 希望降低普通人接近 AI 能力的操作门槛，让复杂的设备能力可以被理解、被授权、被调度，而不是要求每位用户先学会终端、环境变量和部署拓扑。

面向个人的 Agent 设备空间

我们把运行 Operit Core 的一个应用实例称为 CoreNode。一个 CoreNode 可以运行在手机、桌面、浏览器或云端设备上。它不是远程客户端的影子，也不是某个中心服务的附属进程。每个节点都有自己的 HostManager，由本机 Host 提供文件、终端、浏览器、网络、模型、存储和平台交互能力。CoreNode 的角色由可用性、能力和用户策略决定，而不是由操作系统或设备身份预先决定。



图 1 个人 Space 中平等 CoreNode 的协作关系



Space 是多个已经建立信任关系的 CoreNode 所组成的协作范围。它首先解决的是哪些节点共享持续同步的业务记录，哪些节点能够承载路由流量。它不是一台中心服务器，也不是把远端的文件系统、管理员权限或正在运行的进程直接暴露给其他节点。节点的本地能力始终由它自己的 Host 管理；跨节点协作是让合适的节点在合适的上下文中执行，而不是把所有能力复制到发起端。

概念	在个人空间中的职责	必须保持的边界
CoreNode	运行 Core 与本地 Host 的独立节点，可以承担对话、工具或持续任务。	节点之间地位对等，长期在线不等于中心化。
Host	提供本地文件、终端、浏览器、网络、模型与系统接口。	系统权限由本机和部署环境决定，不会随同步被继承。
Space	组织成员、持久化同步范围与可路由的协作关系。	不替代本地执行环境，也不等同于共享 root 权限。
Link 和 PeerLink	处理配对、认证、实时调用、观察流和多跳转发。	两两信任是直接网络连接的基础。
Binding	记录某项可执行工作下一段应该由哪个节点继续。	改变执行位置，不把聊天或 Agent 数据搬迁成单点所有。

Operit2 的工程架构

Operit2 的工程结构从 Rust Core 出发，再由平台 Host、Flutter 应用、CLI 和 Web Access 接入。Core workspace 将 Host API、基础模型、持久化存储、工具与插件 SDK、模型 Provider、JavaScript bridge、运行时、节点路由、Access、Link 与代理生成拆分为明确的 crate。父运行时负责把具体实现装配在一起，低层 crate 通过数据结构和 trait 暴露能力，避免平台实现、模型适配和前端入口相互缠绕。[1]

这种拆分不是为了堆叠抽象，而是为了让不同变化速度的部分拥有不同的演进节奏。模型 Provider、Prompt、工具规划、UI 和脚本运行时快速变化；Host API、存储边界、节点身份、同步记录和权限语义需要更谨慎地变化。Core crate 的依赖方向将 Host API、数据模型与存储放在下层，将工具、Provider、JavaScript bridge 和最终运行时分层组合。这为后续替换模型、增加平台或重构交互面保留了空间。

持久化同步与实时传输

在节点协作中，Operit2 有意把两条数据路径分开。SpacePersistenceSyncService 订阅本地持久化变更，在直接配对的节点之间交换同步操作、向量时钟和必要的内容寻址文件。聊天、偏好、对象数据、



Binding 与运行时文件可以按声明的持久化范围同步。节点身份私钥、两两配对密钥、HostManager 平台状态、本机监听地址和临时交互句柄留在本地。这样的分离让恢复和继续成为可设计的问题，也避免把短暂连接状态误认为可复制的业务事实。

实时路径承担 Core call、watch、push、流事件、Agent continuation 与 Host 交互响应。CoreNodeRouter 根据路由元数据决定请求是在本机执行，还是经由 PeerLink 到达目标 CoreNode。实时传输可以等待持久化同步达到某个时钟位置，但这个屏障只保证目标节点看到足够的任务事实，并不意味着正在运行的模型请求、PTY、浏览器会话或进程句柄被原样迁移。

Host 是能力边界

HostManager 将文件系统、终端、HTTP、浏览器自动化、浏览器会话、存储、密钥保管、运行时调度和本地推理等能力以 trait 方式交给 Core。HostEnvironmentDescriptor 同时描述平台、当前权限、隔离状态、结构化能力和工作区根目录。Windows、Linux、Android、OpenHarmony、Apple 与 Web Host 可以通过这一边界暴露不同能力，而上层 Agent 不需要把平台差异硬编码进对话逻辑。

浏览器在 Operit2 中也不只是一个访问页面。Web Access 可以作为一个已运行节点的浏览器访问面；浏览器平台 Host 则有独立的运行时和能力描述。两者不能混为一谈。前者让用户在浏览器中接近自己的 Agent，后者为浏览器成为独立运行节点留下工程路径。

对话主权 任务执行与上下文连续性

多设备同步最容易犯的错误，是让每个设备把自己的即时输入同时注入同一个模型上下文。这样会损坏叙事顺序，也会让模型无法区分谁正在推进任务。Operit2 采用更清晰的边界：发起设备拥有主对话窗口与用户交互主权；实际执行的节点拥有该任务或子任务的局部工具循环；Space 同步任务状态、结果、产物与可恢复记录。所有设备能够知道任务做到了哪里，但不会把所有实时过程直接回灌到同一段模型上下文。

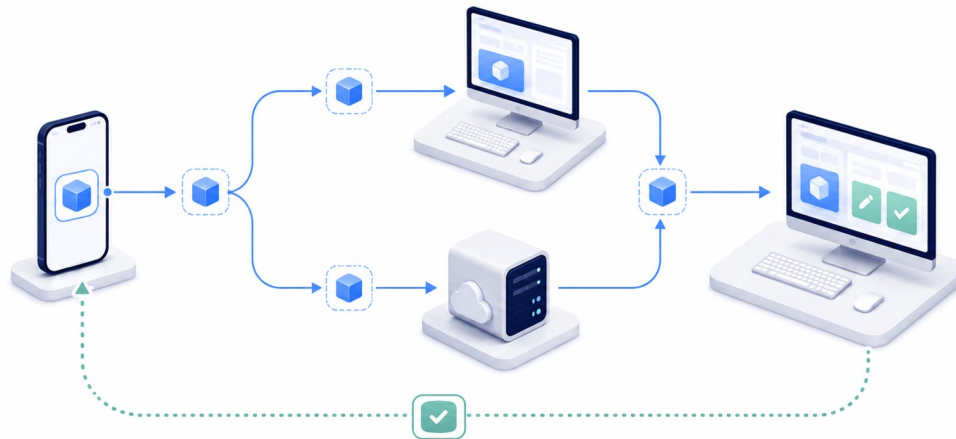


图 2 任务在发起 执行 同步与回看之间保持连续

Binding 是这套边界的执行机制。它用一个不透明 key 记录下一段路由执行由哪个 CoreNode 完成，并带有单调递增的 generation。工具结果先被持久化，随后才可能切换下一轮模型请求。目标节点在看到必要同步记录后继续执行，旧节点不能再推进同一轮。这种单一执行者约束对有副作用的任务尤为重要：一个任务可以被观察、被恢复、被重新调度，但不应在多个节点上悄悄重复提交。

从临时运行留下可恢复的任务事实

一个正在运行的 shell、SSE 流、浏览器标签或模型请求有大量瞬时细节。我们不把无损迁移进程快照当成目标。更有价值的目标是留下足以理解和继续工作的任务事实：任务由哪个节点执行，何时开始，使用了哪些工具参数，完成了哪些主要动作，得到了什么结果，产生了哪些文件或其他产物，发生了哪些需要解释的错误。原始日志可以作为可展开证据保存，语义总结则让下一台设备和下一次对话快速获得判断所需的信息。

信息层	典型内容	在协作与恢复中的作用
实时运行层	进程句柄、PTY、流连接、短期缓存、UI 状态。	由当前节点管理，连接中断时允许结束或重建。
任务轨迹层	工具参数、执行节点、重要事件、结果、错误、产物引用。	可同步、可审计、可在另一节点上继续理解。
语义历史层	阶段总结、用户意图、决定理由、可继续的下一步。	控制主模型上下文密度，支撑备份和恢复。
本地敏感层	设备私钥、配对密钥、Host 临时权限与系统句柄。	保留在节点本地，不作为默认同步内容。



设备能力 健康与调度

Linux 云端设备在个人 Space 中是普通 CoreNode。它可能因为长期在线、算力更强或服务更完整而承担更多工作，但这是一种调度结果，而不是系统赋予它的中心身份。新购买的电脑、新接入的服务器或未来独立运行的浏览器节点，都应该能够加入同一套逻辑：节点拥有自己的能力，用户拥有自己的连续性，任务根据事实被分配。

静态设备标签不足以回答哪个节点更适合执行一项任务。一个节点初次加入时可以声明操作系统、架构、安装的服务、可见硬件、模型运行时和工作区；随后，系统还需要观察它的实时可达性、资源压力、连接质量、执行耗时和失败模式。一次失败只是一条证据，不能永久否定某项能力；一块新 GPU、一个新安装的服务或一次网络环境变化都应更新节点档案。

能力视角	回答的问题	调度含义
声明能力	这台设备声称拥有什么平台、运行时、硬件和服务。	用于初始匹配和兼容性筛选。
实时健康	它现在是否可达、可接收任务、资源是否紧张。	用于短期可用性判断，具有时效性。
长期证据	它曾否成功完成某类工作，耗时与稳定性如何。	形成逐渐收敛的实际能力置信度。
用户策略	隐私、费用、耗电、优先设备、云端使用和冗余偏好。	将效率置于用户选择之后。

冗余是可靠性工具，不是让所有设备长期满载的理由。只读、幂等或明确允许的工作可以做验证性并行、备用切换或子任务拆分；能够修改外部状态的工作需要唯一的执行租约或 generation，并且需要让用户知道动作将发生在哪里。长期来看，Operit2 希望学习的不是一串操作系统名称，而是用户真实使用中的设备生态。

权限 同步与用户主权

个人 Agent 的核心不是替人做所有决定，而是让人能看见、理解并决定重要边界。Operit2 的权限架构将外部真实隔离、Host 被系统授予的能力、AI 当前能力模式和具体工具调用批准从上到下区分开来。下层只能继续收紧能力，不能绕过上层。完整权限不等于 root 或管理员提权；设备进程没有的系统能力，Agent 也不能凭一个应用内开关获得。



层级	责任	用户能理解的含义
运行环境隔离	虚拟机、容器、系统账号、Android 应用沙盒等外部边界。	设备或部署环境本身允许什么。
Host 授权	操作系统或部署环境授予当前 Host 的真实能力。	这台机器和这个账号真正能做什么。
AI 能力模式	只读、工作区读写或完整权限等运行时限制。	此刻 AI 被允许读、写和调用到什么范围。
具体工具批准	对一次有影响的工具调用进行确认。	哪一个动作会在何处发生，是否由我授权。

同步敏感数据时，系统应当提供清楚的对象、目标节点、风险、撤销、轮换和到期语义，而不把命令行和环境变量当成普通用户的前置知识。用户有理由把某个凭据、工作区或私有数据同步到指定节点，也有理由只留在本机。Operit2 的职责是让后果可见，让确认足够具体，并让撤销和恢复成为正常操作；最终的主观能动性属于人。

插件化与可演进的运行时

Agent 的外层形态会快速改变。模型适配器、提示词、工具规划、UI、JavaScript 执行器和应用入口都可能被更好的实现替换。Operit2 选择将这些变化更快的层放入插件、ToolPkg、Skill、MCP、Compose DSL、JavaScript bridge 和 Wasm 执行边界中。仓库已经包含 ToolPkg 包解析与生命周期管理、可发布插件 SDK、JavaScript 执行契约、Wasm ToolPkg 运行时以及面向作者的 TypeScript 声明。[1]

我们把这条原则概括为：把变化插件化，把连续性固化。它并不意味着每一个插件都必须在每一台设备上执行。更可靠的方向是让扩展明确描述自己需要的运行时、Host capability、权限、兼容版本、状态范围和可恢复结果。兼容节点可以执行或接续任务，平台特有则保留其边界。这样，插件的可移动性来自清楚的契约，而不是假设所有设备都相同。

SDK 契约维度	建议描述内容	跨节点价值
身份与兼容性	包标识、版本、来源、Core 与 SDK 兼容范围。	让节点知道是否可以解析、加载和迁移结果。
运行时与依赖	JavaScript、Wasm 或其他运行时，所需服务和资源。	避免把平台假设伪装成通用能力。
能力与权限	Host capability、工具 effect、敏感操作和批准需求。	使调度与用户确认基于真实风险。



SDK 契约维度	建议描述内容	跨节点价值
状态与接续	本地、任务或 Space 状态，产物和恢复信息。	让任务在兼容节点间留下可继续的轨迹。

研究语境与技术立场

大语言模型驱动的 Agent 通常把感知、推理、行动和记忆组合在一个循环中。Xi 等人的综述将 LLM Agent 概括为以模型为核心、与感知和行动相结合的系统；这提供了理解 Agent 的有用框架，但没有替代产品级运行时必须解决的身份、权限、持久化和设备协作问题。[6] Operit2 的位置在于把这些运行时问题作为一等工程对象。

行动轨迹与可解释的任务事实

ReAct 说明，交错的推理轨迹和外部动作可以帮助模型维持任务计划，并让过程比只有最终答案更容易理解。[3] Operit2 不把模型内部思维链当作应该无限同步的对象，却认可任务过程需要留下可验证的外部事实。因此，工具调用、输入、结果、错误、产物、执行节点和阶段总结在系统层面拥有重要地位。它们既服务于当前对话，也服务于跨设备查看、备份恢复和下一次继续。

模型上下文与外部记忆的分层

MemGPT 将有限上下文窗口与分层记忆管理联系起来，指出长对话与大文档需要在不同存储层之间选择信息。[4] Generative Agents 进一步展示了经验记录、反思和动态检索如何影响后续计划。[5] 这些研究启发我们把模型上下文看成稀缺的工作区，而不是用户整个数字生活的唯一载体。Operit2 不宣称解决模型内生的长期记忆问题；它要解决的是模型之外的连续性基础：工作状态、持久化个人与任务历史、设备世界状态、权限与数据所有权记录。

这种分层也解释了为何多设备协作不能等同于全量 token 同步。主模型只应看到完成当前判断所需的摘要、证据与可访问引用；更细粒度的日志和原始产物应在需要时展开。这样可以减少上下文噪声，保留对错误和结果的追溯能力，并避免把每个临时执行细节永久塞入对话窗口。

模型进步之后仍然需要什么

更强的模型可能会消除今天大量提示词胶水、固定 Agent loop 和手写工具规划。这是我们必须为之设计的变化，而不是需要回避的风险。即使模型能力大幅上升，一个长期陪伴用户的 Agent 仍要知道自己在哪些设备上运行、能访问什么、谁授权了何种动作、一次任务留下了什么结果、设备更换后如何恢复。harness 的形态可以变薄，用户连续性的基础设施仍然需要被认真保存。



可靠性 迁移与预览期演进

Operit2 处于基础设施重构与稳定性打磨并行的阶段。我们不会用“永远兼容”阻止必要的架构改进，也不会把每一次重构看成可以无声丢失用户连续性的理由。接口、数据格式、插件契约和同步语义应当可版本化；当迁移发生时，用户应能够理解迁移内容、保留可恢复记录，并知道任务与设备关系是否仍然成立。

对个人用户而言，迁移不只是导出一个配置文件。新手机、新电脑或新服务器加入后，应能恢复用户的 Agent 世界：对话和任务历史、必要的持久化资料、扩展状态、设备关系和可解释的任务轨迹。某个设备损坏、离线或被替换，不应让用户与 Agent 之间的连续性悄悄消失。

可靠性需要用可观察的行为来衡量。长连接的重连与释放、取消传播、背压、流结束、任务去重、内容校验、失败转移、执行 generation 和恢复记录都属于需要长期验证的运行时品质。Rust 为 Core 提供了适合构建长运行系统的语言与生态基础，但语言本身不自动保证没有资源泄漏或生命周期问题；最终品质来自明确的生命周期、背压、取消、资源控制和可观测性。

实施重点与结语

接下来的工程重点应当服务于同一个用户承诺：用户可以在任何合适的设备上发起、查看、批准、继续和恢复自己的 Agent 工作，而不必把设备差异、上下文割裂和复杂配置全部承担在自己身上。为了把这一承诺变成可靠体验，Operit2 将优先收敛以下工作。

优先方向	要解决的真实问题	预期形成的基础能力
能力编排	跨设备暴露的能力还需要成为可理解、可选择、可执行的工具链。	节点可以基于契约、健康、历史证据与用户策略被选择。
长期运行品质	连续任务、流、连接、取消与资源释放需要经受长时间使用。	任务稳定性、可观测性和可恢复性得到系统化验证。
迁移与恢复	新设备加入或旧设备退出时，用户连续性不能依赖手工搬运。	同步、备份、恢复和选择性重建拥有清楚的用户路径。
插件 SDK	扩展作者需要知道如何声明能力、风险、状态和兼容性。	兼容节点可以执行或接续扩展工作，而不承诺伪通用性。
人机授权体验	普通用户需要理解敏感同步和有副作用操作。	确认、撤销、轮换和策略选择成为自然交互，而非命令行负担。



Operit2 不是要把人的所有设备变成同一台机器。它要让每台设备继续保有自己擅长的能力，同时让用户拥有一条不断延续的 Agent 关系。手机可以陪伴，桌面可以创作，云端设备可以持续工作，浏览器可以随时接近结果。Agent 的价值不应被某一台设备、某一次登录或某一段上下文截断。我们愿意为这件事持续重构底层，也愿意用清楚的边界、可恢复的数据和长期工程投入把它做好。



附录 源码基线与参考资料

源码基线

下表列出本白皮书写作时用于校对工程叙事的仓库位置。它们不是 API 稳定性承诺，而是帮助读者从概念返回当前实现与设计文档的入口。

主题	源码或文档入口	用于校对的内容
CoreNode Space Binding	docs/core-node-space-binding-architecture.md core/crates/node/runtime/src/CoreNodeRouter.rs	节点平等、Space 范围、Binding 路由和单一执行者语义。
持久化同步	core/crates/node/runtime/src/SpacePersistenceSyncService.rs	变更触发同步、直接配对节点、同步操作和文件传输。
Link 与 Access	docs/link-access-architecture.md	本地应用链路与 Space 节点链路的职责分离。
Host 能力	core/crates/foundation/host-api/src/HostManager.rs core/crates/foundation/host-api/src/lib.rs	HostManager、环境描述、平台能力和运行时 trait。
权限边界	docs/permission-access-architecture.md	运行环境、Host 授权、AI 能力模式与工具批准的分层。
插件运行时	core/CRATE_BOUNDARIES.md plugins/ core/crates/plugin/sdk/	ToolPkg、JavaScript、Wasm、SDK 和运行时边界。
历史起点	D:/Operit/terminal/.../TerminalManager.kt D:/Operit/terminal/.../LocalTerminalProvider.kt D:/Operit/terminal/.../UbuntuDocumentsProvider.kt	Android 上的 Ubuntu PRoot、终端会话与 SAF 文件系统暴露。

参考资料

[1] AAswordman. Operit2 source repository. GitHub. <https://github.com/AAswordman/Operit2> 访问日期 2026 年 9 月 7 日。

[2] AAswordman. Operit source repository. GitHub. <https://github.com/AAswordman/Operit> 访问日期 2026 年 9 月 7 日。

[3] Yao, S., Zhao, J., Yu, D., et al. ReAct Synergizing Reasoning and Acting in Language Models. ICLR 2023. <https://arxiv.org/abs/2210.03629>



- [4] Packer, C., Fang, V., Patil, S. G., et al. MemGPT Towards LLMs as Operating Systems. arXiv 2023.
<https://arxiv.org/abs/2310.08560>
- [5] Park, J. S., O'Brien, J. C., Cai, C. J., et al. Generative Agents Interactive Simulacra of Human Behavior.
UIST 2023. <https://arxiv.org/abs/2304.03442>
- [6] Xi, Z., Chen, W., Guo, X., et al. The Rise and Potential of Large Language Model Based Agents A Survey.
arXiv 2023. <https://arxiv.org/abs/2309.07864>